

Breast Cancer Wisconsin Diagnostic Dataset - Lab 4

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from IPython.display import display
from matplotlib.colors import ListedColormap
from matplotlib.patches import Patch
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score, precision_s
from sklearn.model_selection import StratifiedKFold, cross_val_score, train_test_sp
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder, StandardScaler

sns.set_theme(style='whitegrid')
pd.set_option('display.max_columns', None)
```

Task 1 - Data Preparation

```
In [2]: df_raw = pd.read_csv('brca.csv')

display(df_raw.head())
display(df_raw.tail())
print('Dataset shape:', df_raw.shape)
print('Column names:', list(df_raw.columns))
print('Data types:')
display(df_raw.dtypes.to_frame(name='dtype'))
print('Dataset information:')
df_raw.info()
print('Descriptive statistics:')
display(df_raw.describe())
print('Missing values:')
display(df_raw.isna().sum().to_frame(name='missing_values'))
print('Duplicate records:', df_raw.duplicated().sum())

df_clean = df_raw.rename(columns={df_raw.columns[0]: 'id', 'y': 'diagnosis'}).copy()

if df_clean.isna().sum().sum() > 0:
    numeric_columns = df_clean.select_dtypes(include=[np.number]).columns
    df_clean[numeric_columns] = df_clean[numeric_columns].fillna(df_clean[numeric_c
    categorical_columns = df_clean.select_dtypes(exclude=[np.number]).columns
    for column in categorical_columns:
        df_clean[column] = df_clean[column].fillna(df_clean[column].mode().iloc[0])

duplicate_count = df_clean.duplicated().sum()
df_clean = df_clean.drop_duplicates().reset_index(drop=True)
print('Duplicate records removed:', duplicate_count)

label_encoder = LabelEncoder()
df_clean['diagnosis_encoded'] = label_encoder.fit_transform(df_clean['diagnosis'])
print('Diagnosis encoding:', dict(zip(label_encoder.classes_, label_encoder.transfo
```

```

feature_columns = [column for column in df_clean.columns if column not in ['id', 'diagnosis']]
X = df_clean[feature_columns]
y = df_clean['diagnosis_encoded']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled_df = pd.DataFrame(X_scaled, columns=feature_columns)

print('Scaled feature matrix shape:', X_scaled_df.shape)
print('Feature columns used for KNN:')
print(feature_columns)

```

	Unnamed: 0	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	x.smoothness_mean
0	1	13.540	14.36	87.46	566.3	0.
1	2	13.080	15.71	85.63	520.0	0.
2	3	9.504	12.44	60.34	273.9	0.
3	4	13.030	18.42	82.61	523.8	0.
4	5	8.196	16.84	51.71	201.9	0.

	Unnamed: 0	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	x.smoothness_mean
564	565	20.92	25.09	143.0	1347.0	
565	566	21.56	22.39	142.0	1479.0	
566	567	20.13	28.25	131.2	1261.0	
567	568	16.60	28.08	108.3	858.1	
568	569	20.60	29.33	140.1	1265.0	

Dataset shape: (569, 32)

Column names: ['Unnamed: 0', 'x.radius_mean', 'x.texture_mean', 'x.perimeter_mean', 'x.area_mean', 'x.smoothness_mean', 'x.compactness_mean', 'x.concavity_mean', 'x.concave_pts_mean', 'x.symmetry_mean', 'x.fractal_dim_mean', 'x.radius_se', 'x.texture_se', 'x.perimeter_se', 'x.area_se', 'x.smoothness_se', 'x.compactness_se', 'x.concavity_se', 'x.concave_pts_se', 'x.symmetry_se', 'x.fractal_dim_se', 'x.radius_worst', 'x.texture_worst', 'x.perimeter_worst', 'x.area_worst', 'x.smoothness_worst', 'x.compactness_worst', 'x.concavity_worst', 'x.concave_pts_worst', 'x.symmetry_worst', 'x.fractal_dim_worst', 'y']

Data types:

	dtype
Unnamed: 0	int64
x.radius_mean	float64
x.texture_mean	float64
x.perimeter_mean	float64
x.area_mean	float64
x.smoothness_mean	float64
x.compactness_mean	float64
x.concavity_mean	float64
x.concave_pts_mean	float64
x.symmetry_mean	float64
x.fractal_dim_mean	float64
x.radius_se	float64
x.texture_se	float64
x.perimeter_se	float64
x.area_se	float64
x.smoothness_se	float64
x.compactness_se	float64
x.concavity_se	float64
x.concave_pts_se	float64
x.symmetry_se	float64
x.fractal_dim_se	float64
x.radius_worst	float64
x.texture_worst	float64
x.perimeter_worst	float64
x.area_worst	float64
x.smoothness_worst	float64
x.compactness_worst	float64
x.concavity_worst	float64
x.concave_pts_worst	float64
x.symmetry_worst	float64

dtype**x.fractal_dim_worst** float64**y** str

Dataset information:

<class 'pandas.DataFrame'>

RangeIndex: 569 entries, 0 to 568

Data columns (total 32 columns):

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	569 non-null	int64
1	x.radius_mean	569 non-null	float64
2	x.texture_mean	569 non-null	float64
3	x.perimeter_mean	569 non-null	float64
4	x.area_mean	569 non-null	float64
5	x.smoothness_mean	569 non-null	float64
6	x.compactness_mean	569 non-null	float64
7	x.concavity_mean	569 non-null	float64
8	x.concave_pts_mean	569 non-null	float64
9	x.symmetry_mean	569 non-null	float64
10	x.fractal_dim_mean	569 non-null	float64
11	x.radius_se	569 non-null	float64
12	x.texture_se	569 non-null	float64
13	x.perimeter_se	569 non-null	float64
14	x.area_se	569 non-null	float64
15	x.smoothness_se	569 non-null	float64
16	x.compactness_se	569 non-null	float64
17	x.concavity_se	569 non-null	float64
18	x.concave_pts_se	569 non-null	float64
19	x.symmetry_se	569 non-null	float64
20	x.fractal_dim_se	569 non-null	float64
21	x.radius_worst	569 non-null	float64
22	x.texture_worst	569 non-null	float64
23	x.perimeter_worst	569 non-null	float64
24	x.area_worst	569 non-null	float64
25	x.smoothness_worst	569 non-null	float64
26	x.compactness_worst	569 non-null	float64
27	x.concavity_worst	569 non-null	float64
28	x.concave_pts_worst	569 non-null	float64
29	x.symmetry_worst	569 non-null	float64
30	x.fractal_dim_worst	569 non-null	float64
31	y	569 non-null	str

dtypes: float64(30), int64(1), str(1)

memory usage: 142.4 KB

Descriptive statistics:

	Unnamed: 0	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	x.smooth
count	569.000000	569.000000	569.000000	569.000000	569.000000	
mean	285.000000	14.127292	19.289649	91.969033	654.889104	
std	164.400426	3.524049	4.301036	24.298981	351.914129	
min	1.000000	6.981000	9.710000	43.790000	143.500000	
25%	143.000000	11.700000	16.170000	75.170000	420.300000	
50%	285.000000	13.370000	18.840000	86.240000	551.100000	
75%	427.000000	15.780000	21.800000	104.100000	782.700000	
max	569.000000	28.110000	39.280000	188.500000	2501.000000	

Missing values:

	missing_values
Unnamed: 0	0
x.radius_mean	0
x.texture_mean	0
x.perimeter_mean	0
x.area_mean	0
x.smoothness_mean	0
x.compactness_mean	0
x.concavity_mean	0
x.concave_pts_mean	0
x.symmetry_mean	0
x.fractal_dim_mean	0
x.radius_se	0
x.texture_se	0
x.perimeter_se	0
x.area_se	0
x.smoothness_se	0
x.compactness_se	0
x.concavity_se	0
x.concave_pts_se	0
x.symmetry_se	0
x.fractal_dim_se	0
x.radius_worst	0
x.texture_worst	0
x.perimeter_worst	0
x.area_worst	0
x.smoothness_worst	0
x.compactness_worst	0
x.concavity_worst	0
x.concave_pts_worst	0
x.symmetry_worst	0

	missing_values
x.fractal_dim_worst	0
y	0

Duplicate records: 0

Duplicate records removed: 0

Diagnosis encoding: {'B': np.int64(0), 'M': np.int64(1)}

Scaled feature matrix shape: (569, 30)

Feature columns used for KNN:

```
['x.radius_mean', 'x.texture_mean', 'x.perimeter_mean', 'x.area_mean', 'x.smoothness_mean', 'x.compactness_mean', 'x.concavity_mean', 'x.concave_pts_mean', 'x.symmetry_mean', 'x.fractal_dim_mean', 'x.radius_se', 'x.texture_se', 'x.perimeter_se', 'x.area_se', 'x.smoothness_se', 'x.compactness_se', 'x.concavity_se', 'x.concave_pts_se', 'x.symmetry_se', 'x.fractal_dim_se', 'x.radius_worst', 'x.texture_worst', 'x.perimeter_worst', 'x.area_worst', 'x.smoothness_worst', 'x.compactness_worst', 'x.concavity_worst', 'x.concave_pts_worst', 'x.symmetry_worst', 'x.fractal_dim_worst']
```

Scaled feature matrix shape: (569, 30)

Feature columns used for KNN:

```
['x.radius_mean', 'x.texture_mean', 'x.perimeter_mean', 'x.area_mean', 'x.smoothness_mean', 'x.compactness_mean', 'x.concavity_mean', 'x.concave_pts_mean', 'x.symmetry_mean', 'x.fractal_dim_mean', 'x.radius_se', 'x.texture_se', 'x.perimeter_se', 'x.area_se', 'x.smoothness_se', 'x.compactness_se', 'x.concavity_se', 'x.concave_pts_se', 'x.symmetry_se', 'x.fractal_dim_se', 'x.radius_worst', 'x.texture_worst', 'x.perimeter_worst', 'x.area_worst', 'x.smoothness_worst', 'x.compactness_worst', 'x.concavity_worst', 'x.concave_pts_worst', 'x.symmetry_worst', 'x.fractal_dim_worst']
```

Feature scaling is important for KNN because the algorithm uses distance between points. StandardScaler makes sure no single feature dominates just because it has larger numeric values.

Inference: The dataset was loaded, checked, and prepared for KNN. The target was encoded and the features were scaled so distance-based comparison is fair.

Task 2 - Train-Test Split Analysis

```
In [3]: def evaluate_split(test_size, k_value, random_state=42):
        X_train, X_test, y_train, y_test = train_test_split(
            X_scaled, y, test_size=test_size, random_state=random_state, stratify=y
        )
        model = KNeighborsClassifier(n_neighbors=k_value)
        model.fit(X_train, y_train)
        predictions = model.predict(X_test)
        return {
            'split': f'{int((1 - test_size) * 100)}:{int(test_size * 100)}',
            'train_size': len(X_train),
            'test_size': len(X_test),
            'accuracy': accuracy_score(y_test, predictions)
        }

initial_k = 5
split_results = pd.DataFrame([
    evaluate_split(0.2, initial_k),
```

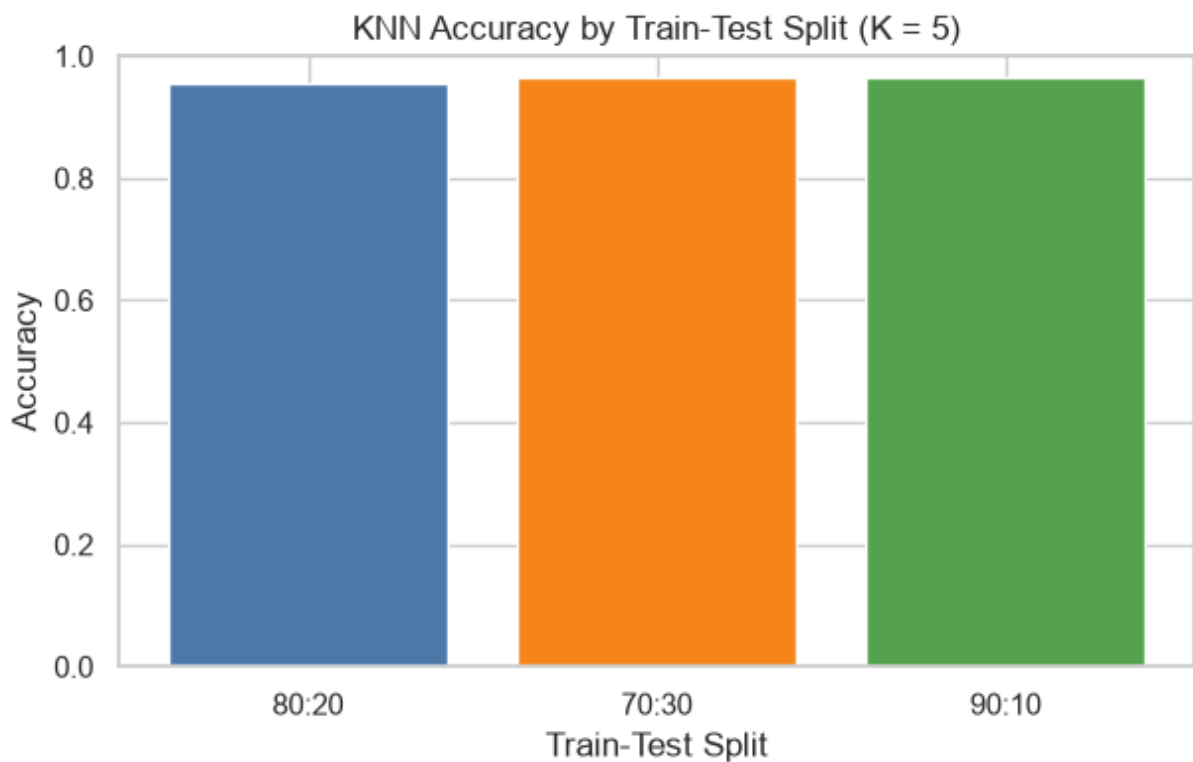
```

    evaluate_split(0.3, initial_k),
    evaluate_split(0.1, initial_k),
])
split_results = split_results[['split', 'train_size', 'test_size', 'accuracy']]
display(split_results)

plt.figure(figsize=(7, 4))
plt.bar(split_results['split'], split_results['accuracy'], color=['#4C78A8', '#F585
plt.title('KNN Accuracy by Train-Test Split (K = 5)')
plt.xlabel('Train-Test Split')
plt.ylabel('Accuracy')
plt.ylim(0, 1)
plt.show()

```

	split	train_size	test_size	accuracy
0	80:20	455	114	0.956140
1	70:30	398	171	0.964912
2	90:10	512	57	0.964912



Inference: The table and bar chart show how the same K value behaves under different split ratios. The best split is the one with the highest accuracy in the output table.

Task 3 - Heuristic K Selection

```

In [4]: X_train_80, X_test_80, y_train_80, y_test_80 = train_test_split(
        X_scaled, y, test_size=0.2, random_state=42, stratify=y
    )

```

```

n_train = len(X_train_80)
raw_k = np.sqrt(n_train)
rounded_k = int(round(raw_k))
if rounded_k % 2 == 1:
    heuristic_k = rounded_k
else:
    lower_k = max(1, rounded_k - 1)
    upper_k = rounded_k + 1
    heuristic_k = lower_k if abs(raw_k - lower_k) <= abs(upper_k - raw_k) else upper_k

print('Training samples:', n_train)
print('Raw sqrt(n):', raw_k)
print('Heuristic K (odd):', heuristic_k)

heuristic_model = KNeighborsClassifier(n_neighbors=heuristic_k)
heuristic_model.fit(X_train_80, y_train_80)
heuristic_predictions = heuristic_model.predict(X_test_80)
heuristic_accuracy = accuracy_score(y_test_80, heuristic_predictions)
print('Heuristic K test accuracy:', heuristic_accuracy)

```

```

Training samples: 455
Raw sqrt(n): 21.330729007701542
Heuristic K (odd): 21
Heuristic K test accuracy: 0.9385964912280702

```

Odd K is usually preferred because it reduces the chance of a tie when the neighbors vote. That makes the prediction decision simpler and more stable.

Inference: The heuristic K was computed from the training size and then adjusted to an odd number. The model was trained with that K and its holdout accuracy was measured.

K Optimization

```

In [5]: k_start = max(1, heuristic_k - 5)
        k_end = heuristic_k + 5
        k_values = list(range(k_start, k_end + 1))

        optimization_rows = []
        for k in k_values:
            model = KNeighborsClassifier(n_neighbors=k)
            model.fit(X_train_80, y_train_80)
            predictions = model.predict(X_test_80)
            optimization_rows.append({
                'k': k,
                'test_accuracy': accuracy_score(y_test_80, predictions)
            })

        optimization_results = pd.DataFrame(optimization_rows)
        display(optimization_results)

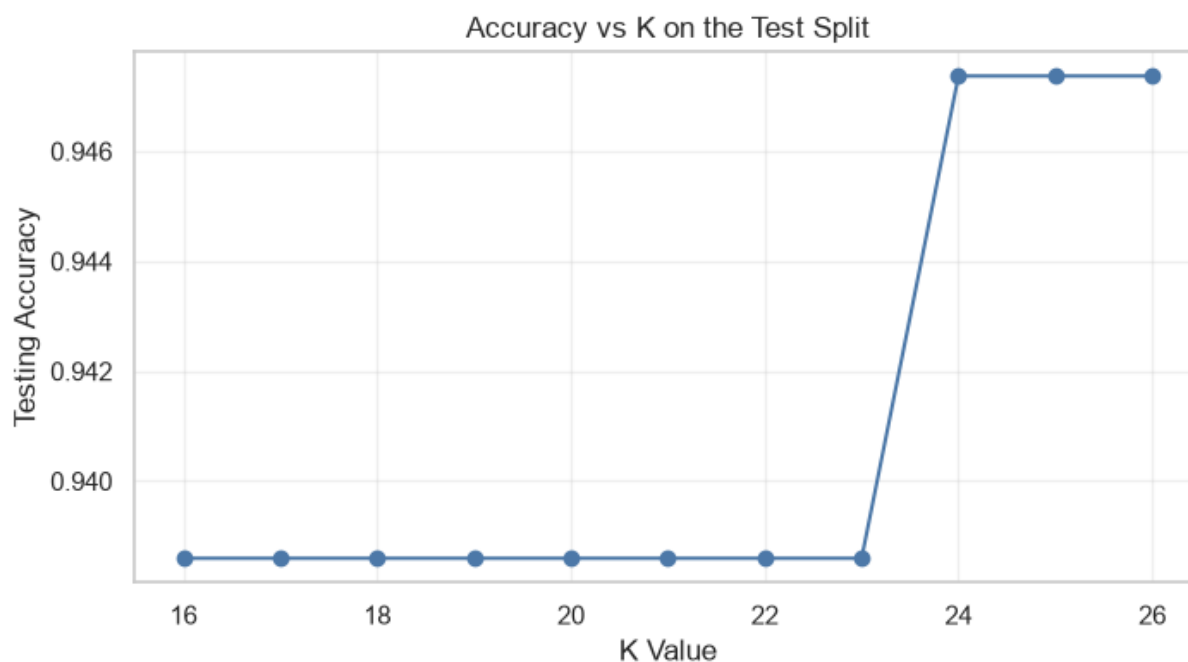
        plt.figure(figsize=(8, 4))
        plt.plot(optimization_results['k'], optimization_results['test_accuracy'], marker='o')
        plt.title('Accuracy vs K on the Test Split')
        plt.xlabel('K Value')
        plt.ylabel('Testing Accuracy')

```

```
plt.grid(True, alpha=0.3)
plt.show()

best_test_row = optimization_results.loc[optimization_results['test_accuracy'].idxmax]
best_k_test = int(best_test_row['k'])
best_test_accuracy = float(best_test_row['test_accuracy'])
print('Best K from testing accuracy:', best_k_test)
print('Best testing accuracy:', best_test_accuracy)
```

	k	test_accuracy
0	16	0.938596
1	17	0.938596
2	18	0.938596
3	19	0.938596
4	20	0.938596
5	21	0.938596
6	22	0.938596
7	23	0.938596
8	24	0.947368
9	25	0.947368
10	26	0.947368



Best K from testing accuracy: 24
Best testing accuracy: 0.9473684210526315

Inference: The accuracy curve shows how performance changes as K grows. The selected K is the one with the highest testing accuracy in the table.

Distance Metrics

Euclidean Distance measures the straight-line distance between two points. It is the most common distance metric for KNN when features are scaled properly. It works well when nearby points in all directions should be treated similarly.

Manhattan Distance measures distance by adding the absolute differences across features. It can be useful when the data has many dimensions or when outliers affect straight-line distance too much. It is often preferred when movement is more grid-like than diagonal.

Euclidean distance is often preferred for smooth numeric data. Manhattan distance is often preferred when the feature space is sparse or noisy.

Decision Boundary

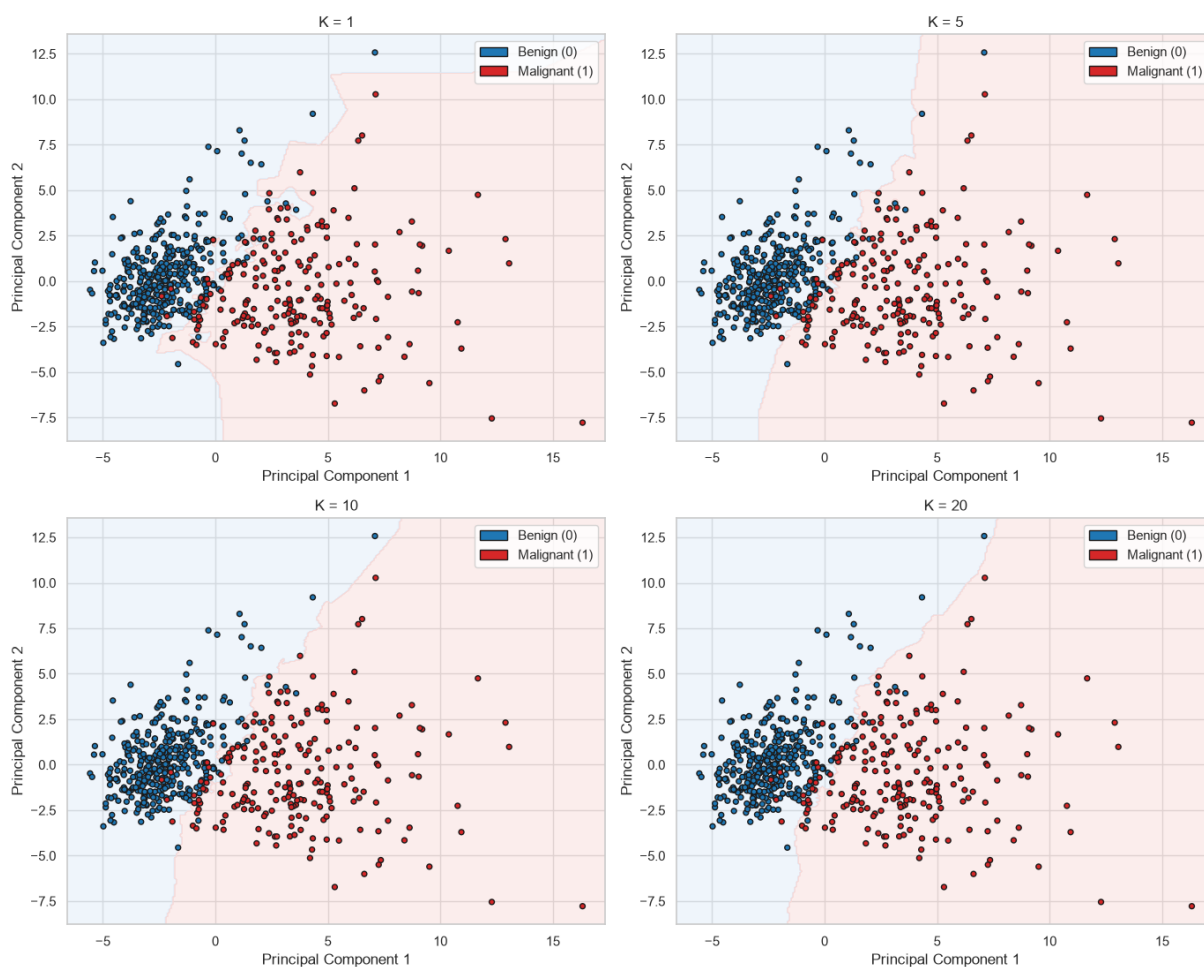
```
In [6]: pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

def plot_boundary(ax, X_data, y_data, k_value):
    model = KNeighborsClassifier(n_neighbors=k_value)
    model.fit(X_data, y_data)
    x_min, x_max = X_data[:, 0].min() - 1, X_data[:, 0].max() + 1
    y_min, y_max = X_data[:, 1].min() - 1, X_data[:, 1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300), np.linspace(y_min, y_max,
    grid_predictions = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    boundary_cmap = ListedColormap(['#dce9f7', '#f8d9d7'])
    point_cmap = ListedColormap(['#1f77b4', '#d62728'])
    ax.contourf(xx, yy, grid_predictions, alpha=0.45, cmap=boundary_cmap)
    ax.scatter(X_data[:, 0], X_data[:, 1], c=y_data, cmap=point_cmap, edgecolor='k')
    ax.set_title(f'K = {k_value}')
    ax.set_xlabel('Principal Component 1')
    ax.set_ylabel('Principal Component 2')
    legend_handles = [
        Patch(facecolor='#1f77b4', edgecolor='k', label='Benign (0)'),
        Patch(facecolor='#d62728', edgecolor='k', label='Malignant (1)')
    ]
    ax.legend(handles=legend_handles, loc='best')

fig, axes = plt.subplots(2, 2, figsize=(14, 12))
for ax, k_value in zip(axes.ravel(), [1, 5, 10, 20]):
    plot_boundary(ax, X_pca, y.values, k_value)

fig.suptitle('KNN Decision Boundaries in PCA Space', fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

KNN Decision Boundaries in PCA Space



For $K = 1$, the boundary is usually the most jagged because each point can strongly influence its neighborhood.

For $K = 5$, the boundary should look smoother and less sensitive to isolated points.

For $K = 10$, the model usually becomes more generalized and the boundary grows more stable.

For $K = 20$, the boundary is typically the smoothest and can start to ignore smaller local patterns.

Task 4 - Cross Validation

```
In [7]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cross_validation_rows = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    cv_scores = cross_val_score(model, X_scaled, y, cv=cv, scoring='accuracy')
    holdout_accuracy = optimization_results.loc[optimization_results['k'] == k, 'te
    cross_validation_rows.append({
        'k': k,
```

```

        'train_test_accuracy': holdout_accuracy,
        'cv_mean_accuracy': cv_scores.mean(),
        'cv_std': cv_scores.std()
    })

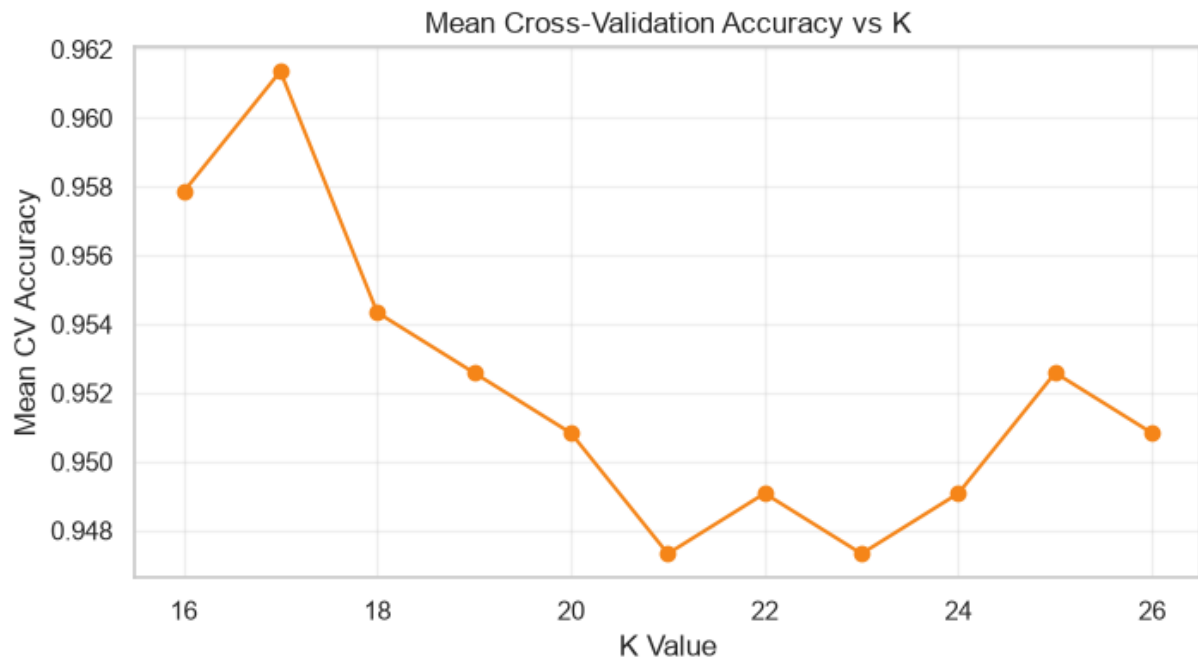
cv_results = pd.DataFrame(cross_validation_rows)
cv_results['combined_score'] = cv_results[['train_test_accuracy', 'cv_mean_accuracy',
display(cv_results)

plt.figure(figsize=(8, 4))
plt.plot(cv_results['k'], cv_results['cv_mean_accuracy'], marker='o', color='#F5851')
plt.title('Mean Cross-Validation Accuracy vs K')
plt.xlabel('K Value')
plt.ylabel('Mean CV Accuracy')
plt.grid(True, alpha=0.3)
plt.show()

final_best_row = cv_results.sort_values(['combined_score', 'cv_mean_accuracy', 'tra
final_best_k = int(final_best_row['k'])
print('Final best K:', final_best_k)
print('Combined score:', float(final_best_row['combined_score']))

```

	k	train_test_accuracy	cv_mean_accuracy	cv_std	combined_score
0	16	0.938596	0.957848	0.012835	0.948222
1	17	0.938596	0.961357	0.011851	0.949977
2	18	0.938596	0.954339	0.017841	0.946468
3	19	0.938596	0.952585	0.020412	0.945591
4	20	0.938596	0.950831	0.018004	0.944714
5	21	0.938596	0.947322	0.022821	0.942959
6	22	0.938596	0.949076	0.020252	0.943836
7	23	0.938596	0.947322	0.022821	0.942959
8	24	0.947368	0.949076	0.020252	0.948222
9	25	0.947368	0.952585	0.019643	0.949977
10	26	0.947368	0.950831	0.018839	0.949100



Final best K: 17

Combined score: 0.9499767116907313

Inference: The comparison table shows both holdout accuracy and 5-fold cross-validation accuracy for each K. The final K is chosen using the strongest overall balance between the two scores.

Task 5 - Final Model Evaluation

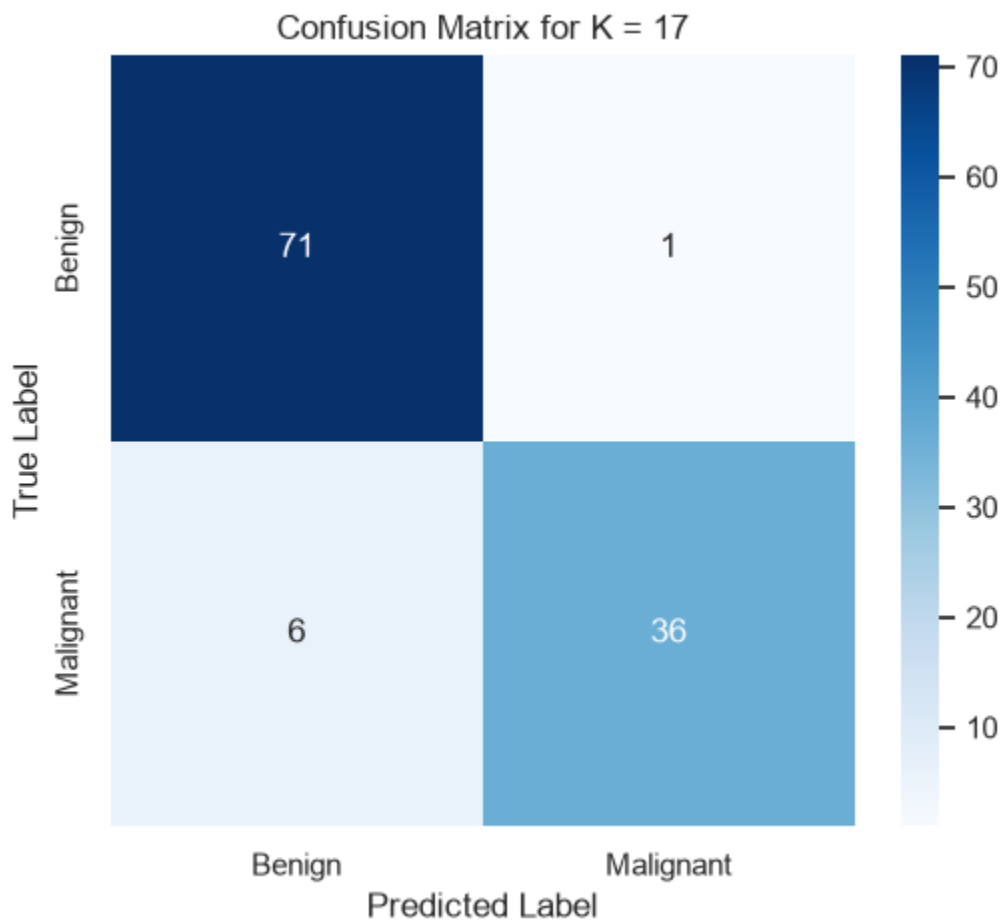
```
In [8]: final_model = KNeighborsClassifier(n_neighbors=final_best_k)
final_model.fit(X_train_80, y_train_80)
final_predictions = final_model.predict(X_test_80)
final_probabilities = final_model.predict_proba(X_test_80)[: , 1]

final_metrics = pd.DataFrame([
    {'Metric': 'Accuracy', 'Value': accuracy_score(y_test_80, final_predictions)},
    {'Metric': 'Precision', 'Value': precision_score(y_test_80, final_predictions)},
    {'Metric': 'Recall', 'Value': recall_score(y_test_80, final_predictions)},
    {'Metric': 'F1 Score', 'Value': f1_score(y_test_80, final_predictions)},
    {'Metric': 'ROC-AUC Score', 'Value': roc_auc_score(y_test_80, final_probabiliti
)])
display(final_metrics)
```

	Metric	Value
0	Accuracy	0.938596
1	Precision	0.972973
2	Recall	0.857143
3	F1 Score	0.911392
4	ROC-AUC Score	0.995040

Inference: The metrics table shows how the final KNN model performs on the holdout test set. Recall and ROC-AUC are especially important to check for cancer detection.

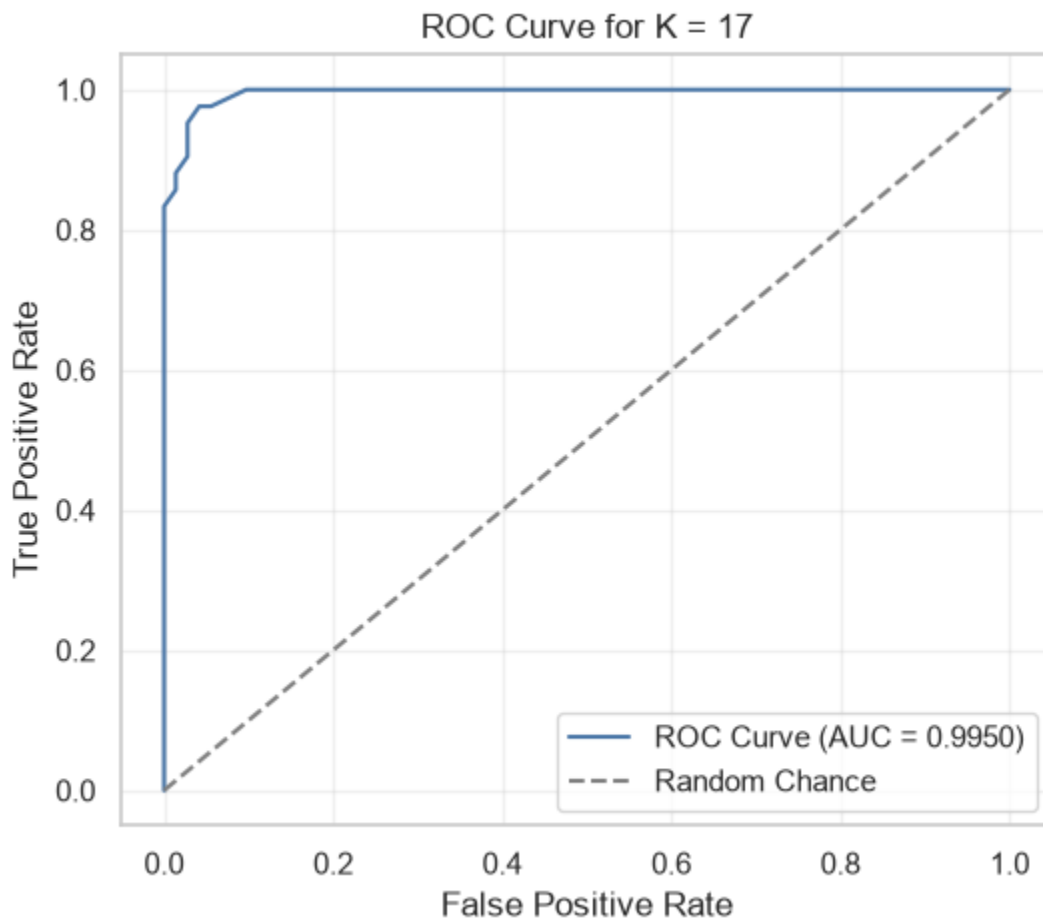
```
In [9]: cm = confusion_matrix(y_test_80, final_predictions)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Benign', 'Malignant'],
            yticklabels=['Benign', 'Malignant'], title=f'Confusion Matrix for K = {final_best_k}')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



Inference: The confusion matrix shows where the model gets predictions right and where it makes mistakes. False negatives are the most serious errors in cancer screening.

```
In [10]: fpr, tpr, thresholds = roc_curve(y_test_80, final_probabilities)
roc_auc = roc_auc_score(y_test_80, final_probabilities)

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color='#4C78A8', label=f'ROC Curve (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Chance')
plt.title(f'ROC Curve for K = {final_best_k}')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.show()
```



Inference: A curve closer to the top-left corner means better classification performance. A higher ROC-AUC means the model separates the two classes more effectively.

Task 6: Comparative Study with Regression (Lab 3 Integration)

Regression evaluation focuses on measuring the magnitude of prediction errors, whereas classification evaluation focuses on whether the predicted class label is correct.

R^2 Score vs Accuracy: R^2 Score measures how well a regression model explains the variation in continuous target values. Accuracy measures the percentage of correctly classified samples. Both indicate overall model performance, but they are used for different prediction tasks.

RMSE vs F1-Score: RMSE measures the average prediction error while giving higher weight to larger errors. F1-Score combines Precision and Recall into a single metric and evaluates the balance between correctly identifying positive cases and avoiding false predictions.

MAE vs Confusion Matrix: MAE calculates the average absolute difference between predicted and actual values in regression. The Confusion Matrix summarizes classification outcomes by showing True Positives, True Negatives, False Positives, and False Negatives, providing detailed insight into prediction correctness.

Continuous Prediction vs Classification: Regression predicts continuous numerical values such as GPA or medical costs, and its performance is evaluated based on prediction errors. Classification predicts discrete class labels such as benign or malignant, and its performance is evaluated based on correct and incorrect decisions.

Inference

Regression metrics such as MAE, MSE, RMSE, and R^2 Score evaluate how close predicted values are to the actual continuous values. Lower MAE, MSE, and RMSE indicate better prediction accuracy, while a higher R^2 Score indicates that the model explains more variation in the data. These metrics measure the magnitude of prediction errors rather than whether a decision is correct. This evaluation approach was used in Lab 3 to assess Linear Regression performance.

Classification metrics evaluate the correctness of predicted class labels instead of numerical errors. Accuracy measures overall correctness, Precision measures the correctness of positive predictions, Recall measures the ability to detect actual positive cases, F1-Score balances Precision and Recall, and ROC-AUC measures the model's ability to distinguish between classes across different thresholds.

Accuracy alone is insufficient for medical diagnosis because a model may achieve high accuracy while still missing patients with cancer. False negatives are more critical than false positives in healthcare. Therefore, Recall is essential because it minimizes missed cancer cases, and ROC-AUC provides a threshold-independent measure of the model's discrimination ability. Overall, regression evaluation emphasizes prediction error, whereas classification evaluation emphasizes decision quality and reliable identification of class labels.

Task 7 - Analytical Questions

- 1. Why is KNN called a lazy learning algorithm?** KNN does not build a separate training model during the learning stage. It stores the training data and waits until prediction time to compute distances. That is why it is called lazy learning.
- 2. Why is feature scaling required in KNN?** KNN depends on distance, so features with larger numeric ranges can dominate the result. Scaling puts every feature on a similar scale. This makes the distance calculation fair.

3. **Explain heuristic K selection using $\sqrt{n}$.** A common rule is to start with K equal to the square root of the number of training samples. This gives a reasonable balance between noise sensitivity and smoothing. The value is often adjusted to an odd number to avoid ties.
4. **Why is cross-validation more reliable than a single train-test split?** Cross-validation tests the model on several different folds of the data. This gives a more stable estimate of performance. A single split can depend too much on one random division.
5. **How does K affect the bias-variance trade-off?** Small K values usually give low bias and high variance. Large K values usually give higher bias and lower variance. The best K gives a balanced result.
6. **Why is Recall more important than Accuracy in cancer prediction?** Recall checks how many actual cancer cases are found by the model. Missing a real cancer case can be very dangerous. That is why recall is often more important than accuracy in this task.
7. **What is the limitation of very large K values?** Very large K values make the model too smooth. It can ignore useful local patterns in the data. This can lead to underfitting and weaker predictions.

Conclusion

The dataset was loaded, checked for missing values and duplicates, encoded, and scaled for KNN. Feature scaling helped the distance-based model treat all features more fairly. The train-test split and K experiments showed which setup performed best on the holdout data.

The final K value was selected using both holdout testing and cross-validation. The final model was then evaluated with accuracy, precision, recall, F1 score, ROC-AUC, a confusion matrix, and an ROC curve. Recall and ROC-AUC are especially important here because cancer screening should avoid missing positive cases.

Regression evaluation focuses on prediction error, while classification evaluation focuses on correct class decisions. In medical diagnosis, classification metrics are more useful because the goal is to detect the disease case correctly.